L2 Informatique
UE S-E06-0207

Partie I
PPE 3

Programmation des Microcontrôleurs

AVIGNON UNIVERSITÉ

E.Zouania

Plan du Cours

- **Timer/Counter 2 de l'ATMega328**
  - ✓ Registres et configuration du Timer2
  - ✓ Exemples de mise en œuvre du Timer2
  - ✓ Interruptions Liées aux Timers
- **Node RED**
  - ✓ Programmation visuel (CodeLess partielle)
  - ✓ Communication Serial
  - ✓ Supervision (accès aux périphériques via IHM)

Rappel:
CC1 le 12/11/26 14:30-16h évaluation écrite
CC2 du 12/11/26 au 19/11/26 TP d'évaluation

L2 Informatique
UE S-E06-0207

Timer/Counter 2 de l'ATMega328

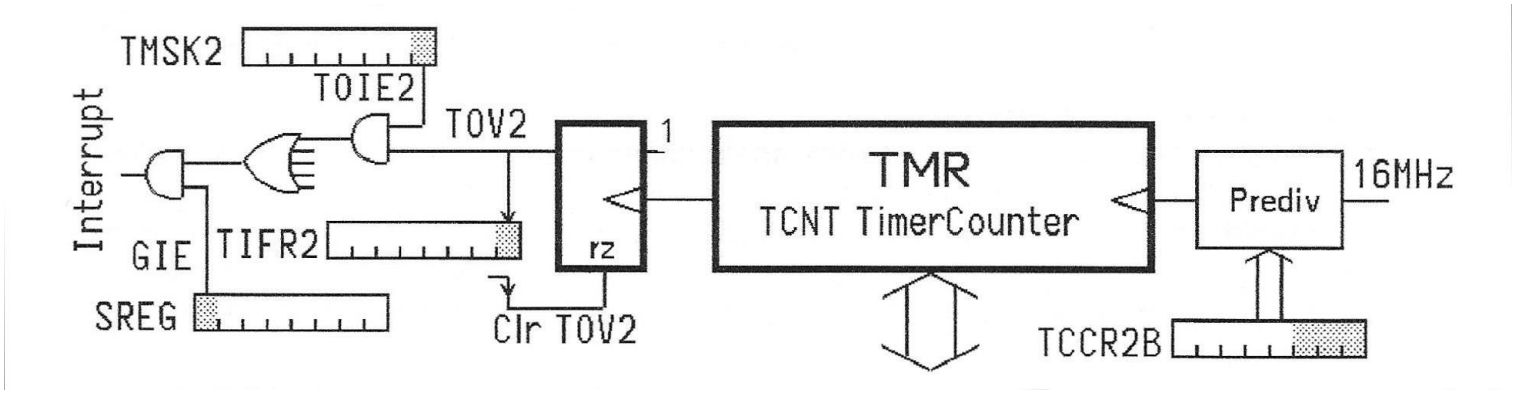
AVIGNON UNIVERSITÉ

E.Zouania

Architecture du contrôle du timer2

(Idem pour Timer0 et Timer1)

Rappel: Toutes les demandes d'interruption arrivent sur une porte OU et sont éventuellement bloquées par le signal ***GIE*** (Global Interrupt Enable) activé par la fonction sei().



Description des registres sur la diapositive suivante

https://www.locoduino.org

❑ **Présentation.**

Un **timer** est un registre interne (périphérique) au micro-contrôleur ATMega328.
C'est en fait un compteur qui s'incrémente (ou se décrémente) à chaque impulsion d'un signal d'horloge (tick), interne ou externe.
Ainsi la mesure du temps qui s 'écoule est possible. D'où le nom donné à ce type de périphérique **Timer/Counter**.
Le temps que met ce **Timer/Counter** pour compter (ou décompter) dépend de la fréquence (ou de la période) des ticks. Le comportement du **Timer/Counter** est directement lié à la programmation de registres de contrôle, accessibles comme d'habitude en lecture/écriture.

❑ **Résumé pratique.**

Si le registre **timer** est de taille 8bits, il est capable de compter de 0 à 255. Lorsque qu'il arrive à 255 il repasse à 0 (2^n=0). On dit qu'il a débordé(overflow). Un flag particulier d'un registre de contrôle propre au **timer** détecte ce débordement en passant de 0 à 1.
Afin de « ralentir » la durée de comptage le Timer/Counter dispose de pré-diviseurs programmables (prescalers).

❑ **Intérêt pratique.**

Un timer utilisé par `millis()`, compte sans cesse en tâche de fond, ce qui ne bloque pas le programme en cours d'exécution. Ce qui n'est pas le cas de la fonction `delay()` qui est bloquante.

❑ **Les 3 modules Timers de l'ATMega328**

- Le **Timer0** de taille 8bits. Utilisé entre autres par les fonctions `delay()`, `millis()` et `micros()`.
- **Le Timer1** de taille 16 bits. Utilisé pour produire des signaux de commande de servomoteurs…(PWM sur pins 9 et 10 de la Uno)
- Le **Timer2** de taille 8bits (PWM sur pins 3 et 11 de la Uno).

Question_1.

Le quartz de l'ATMega328 ayant une fréquence F_Q= **16MHz**, calculer le temps t_{ov1} et t_{ov2} mis par les **Timer1** et **Timer2** respectivement pour déborder.

Réponse:

$T_Q=1/F_Q$ ou T_Q représente la période des « ticks ».

Ainsi $T_Q=62,5.10^{-9}s=62,5ns$ et donc:

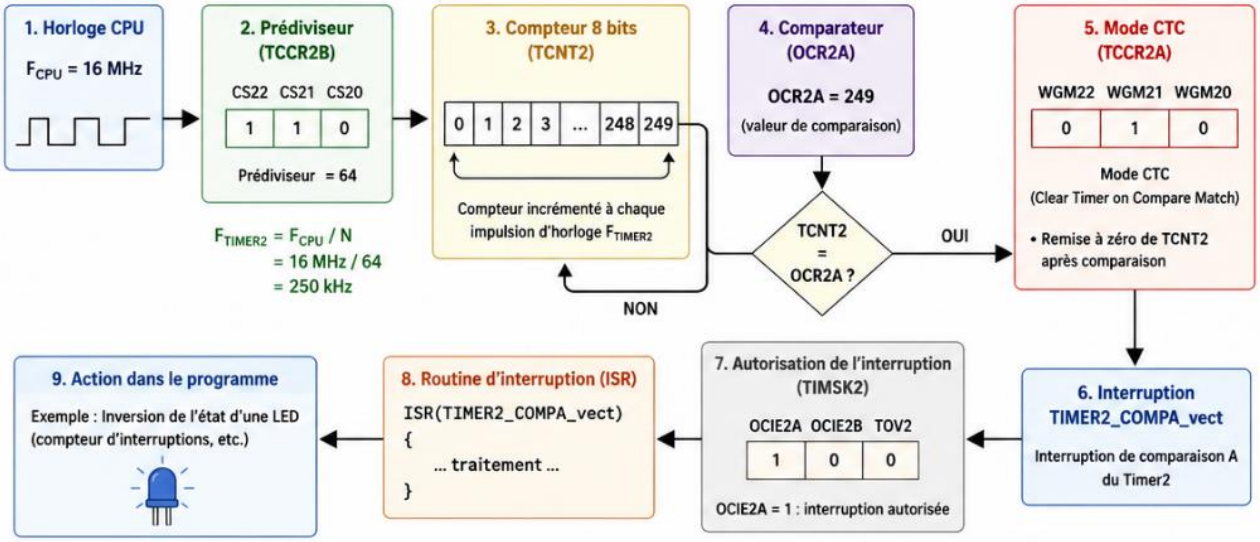
- $t_{ov1}=65536 \times T_Q=4,096ms$
- $t_{ov2}=256 \times T_Q=16\mu s$

AVIGNON UNIVERSITÉ

L2 Informatique
UE S-E06-0207

TIMER2 de l'ATmega328P – Fonctionnement et calcul du temps

Timer2 est un timer 8 bits. En mode CTC (Clear Timer on Compare Match), une interruption est générée lorsque le compteur TCNT2 atteint la valeur définie dans OCR2A.



LES REGISTRES DU TIMER2

TCCR2A	Registre de contrôle A • Bit WGM21 : sélection du mode CTC
TCCR2B	Registre de contrôle B • Bits CS22..0 : sélection du prédiviseur (source d'horloge du timer) • Bit WGM22 : avec TCCR2A, définit le mode de fonctionnement
TCNT2	Compteur 8 bits • Compte les impulsions d'horloge après prédiviseur
OCR2A	Registre de comparaison A • Valeur à atteindre pour déclencher la comparaison A
TIMSK2	Masque d'interruptions Timer2 • Bit OCIE2A : autorise l'interruption sur comparaison A • Bit OCIE2B : comparaison B • Bit TOIE2 : débordement (overflow)
TIFR2	Drapeaux d'interruptions Timer2 • Bit OCF2A : drapeau comparaison A • Bit OCF2B : comparaison B • Bit TOV2 : débordement

CALCUL DE LA PÉRIODE D'INTERRUPTION

Formule générale (mode CTC avec OCR2A)

$$T_{interruption} = \frac{(OCR2A + 1) \times N}{F_{CPU}}$$

Avec :
- F_{CPU} : fréquence CPU (Hz)
- **N** : prédiviseur (1, 8, 32, 64, 128, 256, 1024)
- **OCR2A** : valeur de comparaison (0 → 255)

Exemple du cours :

F_{CPU} = 16 MHz
N = 64
OCR2A = 249

$$T_{interruption} = \frac{(249 + 1) \times 64}{16\,000\,000}$$

$T_{interruption}$ = 0,001 s = 1 ms

CHOIX DU PRÉDIVISEUR (TCCR2B : CS22..0)

CS22	CS21	CS20	Prédiviseur (N)	$F_{TIMER2} = F_{CPU} / N$	$T_{TIMER2} = 1/F_{TIMER2}$
0	0	0	Arrêt du timer	–	–
0	0	1	1	16 MHz	62,5 ns
0	1	0	8	2 MHz	0,5 µs
0	1	1	32	500 kHz	2 µs
1	0	0	64	250 kHz	4 µs
1	0	1	128	125 kHz	8 µs
1	1	0	256	62,5 kHz	16 µs
1	1	1	1024	15,625 kHz	64 µs

À RETENIR
- Le Timer2 transforme l'horloge CPU en une base de temps programmable.
- En mode CTC, l'interruption est générée à chaque égalité **TCNT2 = OCR2A.**
- La période d'interruption dépend du prédiviseur (TCCR2B) et de la valeur **OCR2A.**
- L'interruption n'est effective que si elle est autorisée dans **TIMSK2 (OCIE2A = 1).**

AVIGNON UNIVERSITÉ

- Les registres de contrôle.

Nous nous intéresserons au seul timer2. (Timer0 mêmes registres…)

TCNT2 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
(0xB2)	TCNT2[7:0]								TCNT2
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

TCCR2A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
(0xB0)	COM2A1	COM2A0	COM2B1	COM2B0	–	–	WGM21	WGM20	TCCR2A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	

TCCR2B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
(0xB1)	FOC2A	FOC2B	–	–	WGM22	CS22	CS21	CS20	TCCR2B
Read/Write	W	W	R	R	R	R	R/W	R/W	

TIFR2 – Timer/Counter2 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x17 (0x37)	–	–	–	–	–	OCF2B	OCF2A	TOV2	TIFR2
Read/Write	R	R	R	R	R	R/W	R/W	R/W	

TIMSK2 – Timer/Counter2 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x70)	–	–	–	–	–	OCIE2B	OCIE2A	TOIE2	TIMSK2
Read/Write	R	R	R	R	R	R/W	R/W	R/W	

Détection et ISR sur débordement (TIMER2_OVF_vect)

Mode normal de fonctionnement en « temporisateur » (timer).

- Le registre **TCNT2** est incrémenté de 1 à chaque événement de comptage.
- Le registre ne revient à 0 qu'après un débordement (passage de 0xFF à 0x00).

→ **Configuration de ce mode via les registres TCCR2A et TCCR2B**

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on (1)(2)
0	0	0	0	Normal	0xFF	Immediate	MAX

Prescaler

En temporisateur, le registre de comptage **TCNT2** est incrémenté en fonction des cycles horloge.
L'incrément peut être à chaque cycle horloge (« tick ») (pas de prescaler) ou bien à une fréquence moindre (prescaler nécessaire).
*Rappelons que le cycle horloge est de **1/16 µseconde**. On peut donc mettre un diviseur de fréquence entre l'horloge (16Mhz) et l'incrément du registre de comptage.*

Table 17-9. Clock Select Bit Description

CS22	CS21	CS20	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk_{T2S}/(No prescaling)
0	1	0	clk_{T2S}/8 (From prescaler)
0	1	1	clk_{T2S}/32 (From prescaler)
1	0	0	clk_{T2S}/64 (From prescaler)
1	0	1	clk_{T2S}/128 (From prescaler)
1	1	0	clk_{T2S}/256 (From prescaler)
1	1	1	clk_{T2S}/1024 (From prescaler)

- **Utilisation du Timer2 sans Interruption.**
  - **Exo1_cours1c_Part2.**

Ecrire un « code mixte » permettant de faire clignoter la LED de la carte Arduino Uno à une fréquence de 1Hz avec un rapport cyclique de 50%.

```
const byte Led=13;// Led UNO
#define LedToggle digitalWrite (Led, !digitalRead(Led)) // Une Macro.
byte cpt_Ovf=0; // Compteur de débordement

void setup()
{
  pinMode(Led,OUTPUT);
  bitClear(TCCR2A, WGM20); // bit WGM20=0 (mode "normal")
  bitClear(TCCR2A, WGM21); // bit WGM20=0 (mode "normal")
  TCCR2B = 0b00000110; // Clk/256 (16µs) et WGM22=0 (mode "normal")
  TIFR2 = 0b00000001;// TOV2=1 initialisation du bit de débordement.
  TCNT2 = 6; // Chargement du compteur avec 6 (1 débordement toutes les 4ms)
}

void loop()
{
  if (bitRead(TIFR2,0)==1) // test si TOV2=1 débordement
  {
  TCNT2=6;//Rechargement TCNT2
  bitSet(TIFR2, TOV2); // RàZ Flag overflow nécessaire ici!.
    if (cpt_Ovf++ ==125)
    {
    cpt_Ovf=0;
    LedToggle; // Appel de la macro
    }
  }
}
```

$$f_{Timer2} = \frac{16\,MHz}{256} = 62\,500\,Hz \qquad T = \frac{1}{62500} = 16\,\mu s$$

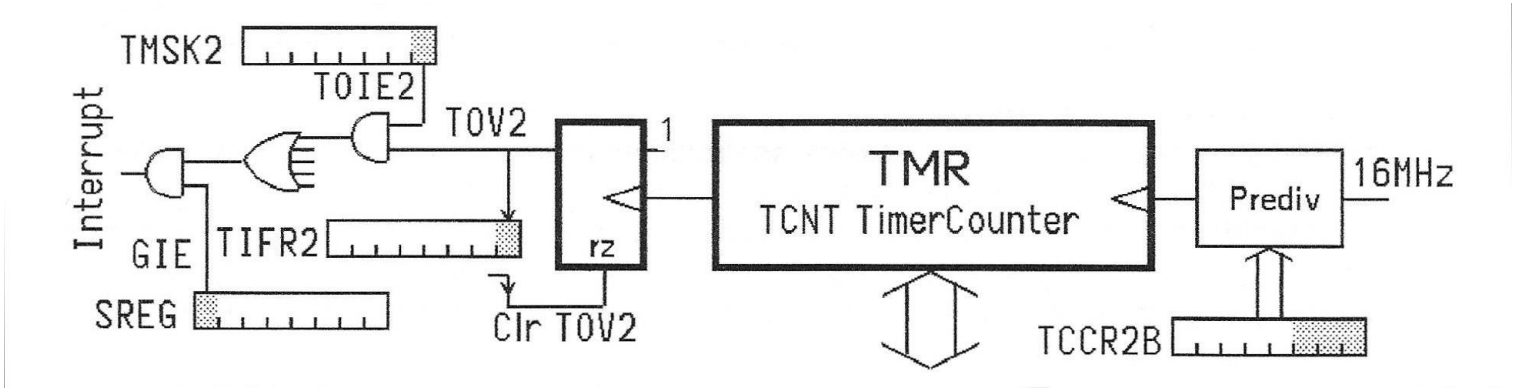
$$256 - 6 = 250$$

$$250 \times 16\,\mu s = 4\,ms$$

$$125 \times 4\,ms = 500\,ms$$

- **Utilisation du Timer2 sans Interruption.**
  - **Exo2.**

Ecrire un programme permettant de faire clignoter la LED de la carte Arduino Uno à une fréquence de 1Hz avec un rapport cyclique de 50% avec une période de comptage de 1ms.



Prediv = ?

F_{Timer2} = ?

T=?

N_{CNT} = ?

Temps Interruption = ?

Utilisation du Timer2 sans Interruption.

Exo2.

Ecrire un permettant de faire clignoter la LED de la carte Arduino Uno à une fréquence de 1Hz avec un rapport cyclique de 50% avec une période de comptage de 1ms.

```
const byte Led = 13;        // LED de la carte UNO
byte cpt_Ovf = 0;           // Compteur de débordements
void LedToggle()
{
  digitalWrite(Led, !digitalRead(Led));
}
void setup()
{
  pinMode(Led, OUTPUT);
  // Timer2 en mode normal
  // WGM21 = 0 et WGM20 = 0
  TCCR2A = _________;
  // Prescaler = 256
  // Fréquence du timer : 16 MHz / 256
  TCCR2B = _________;
  // Initialisation du flag de débordement TOV2
  TIFR2 = _________;
  // Pour obtenir un débordement toutes les 1 ms :
  // période d'un coup d'horloge Timer2 = 16 µs
  // nombre de coups nécessaires = 1 ms / 16 µs = 62,5
  // Choisir une valeur entière permettant d'obtenir environ 1 ms.
  TCNT2 = _________;
}
void loop()
{
  // Tester si le Timer2 a débordé
  if (_____________________________)
  {
```

- **Utilisation du Timer2 sans Interruption.**
  - **Exo2.**

Ecrire un permettant de faire clignoter la LED de la carte Arduino Uno à une fréquence de 1Hz avec un rapport cyclique de 50% avec une période de comptage de 1ms.

```
void loop()
{
  // Tester si le Timer2 a débordé
  if (_____________________________)
  {
    // Recharger le compteur
    TCNT2 = _________;
    // Remettre à zéro le flag TOV2
    TIFR2 = _________;
    // 500 débordements correspondent à 0,5 s
    if (_____________________________)
    {
      cpt_Ovf = 0;
      // Inverser l'état de la LED
      _____________________________;
    }
  }
}
```

- **Utilisation du Timer2 avec Interruption.**
  - **Exo2_cours1c_Part2.**

Ecrire un code permettant de faire clignoter la LED de la carte Arduino Uno à une fréquence de 1Hz avec un rapport cyclique de 50% en utilisant le Timer2 en interruption.

Traitement de l'ISR Timer2 par Overflow.

```
// ARDUINO UNO - IT Timer2 Overflow
const byte Led = 13;
volatile byte cpt_Ovf = 0;     // Compteur d'IT

// Fonction de traitement de l'interruption
// Timer2 Overflow
ISR(TIMER2_OVF_vect)
{
  cpt_Ovf++;
  if (cpt_Ovf == 125)
  {
    // Basculement de la LED
    digitalWrite(Led, !digitalRead(Led));
    cpt_Ovf = 0;
    TCNT2 = 6;                  // Rechargement du Timer2
  }
}
```

Le flag TOV2 est automatiquement remis à 0.

Configuration des registres.

```
void setup()
{
  // Configuration de la LED
  pinMode(Led, OUTPUT);
  digitalWrite(Led, LOW);


  cli();


  // Configuration Timer2
  TCCR2A = 0;                    // Mode Normal (pas PWM)
  TCCR2B = 0x06;                 // Prescaler 256 (Clock/256)
  TIMSK2 = 0x01;                 // IT Timer2 Overflow activé


  TCNT2 = 6;                     // Valeur initiale du compteur


  sei();                          // Activation des interruptions
}


void loop()
{
  // Aucun traitement particulier
}
```

IHM avec Node-RED

Supervision & Pilotage à Distance via Liaison Série Arduino

AVIGNON UNIVERSITÉ

E.Zouania

Objectifs & Architecture IHM

Environnement Node-RED

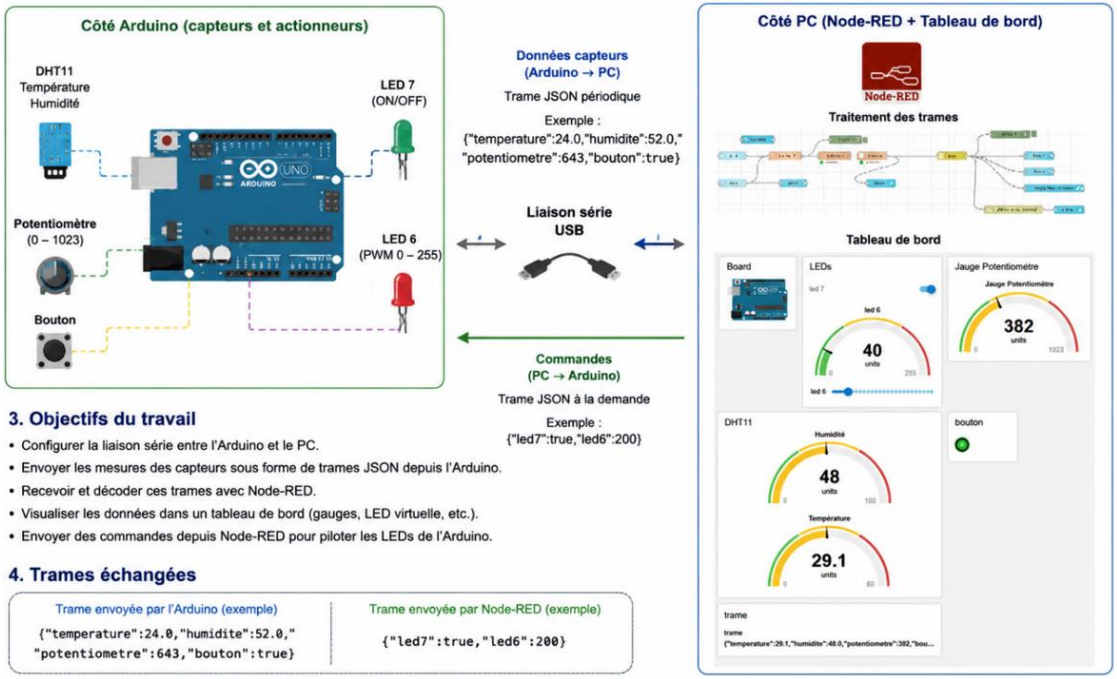
Prise en main de l'outil visuel low-code de création de flux et déploiement du tableau de bord (Dashboard 2.0).

Liaison Série USB

Établissement d'une communication série bidirectionnelle temps réel entre la carte Arduino Uno et Node-RED.

Supervision & Ordres

Affichage des mesures capteurs (DHT11, Potentiomètre) et envoi de commandes vers actionneurs (LED TOR, PWM).



3. Objectifs du travail

- Configurer la liaison série entre l'Arduino et le PC.
- Envoyer les mesures des capteurs sous forme de trames JSON depuis l'Arduino.
- Recevoir et décoder ces trames avec Node-RED.
- Visualiser les données dans un tableau de bord (gauges, LED virtuelle, etc.).
- Envoyer des commandes depuis Node-RED pour piloter les LEDs de l'Arduino.

4. Trames échangées

Trame envoyée par l'Arduino (exemple)	Trame envoyée par Node-RED (exemple)
`{"temperature":24.0,"humidite":52.0,"potentiometre":643,"bouton":true}`	`{"led7":true,"led6":200}`

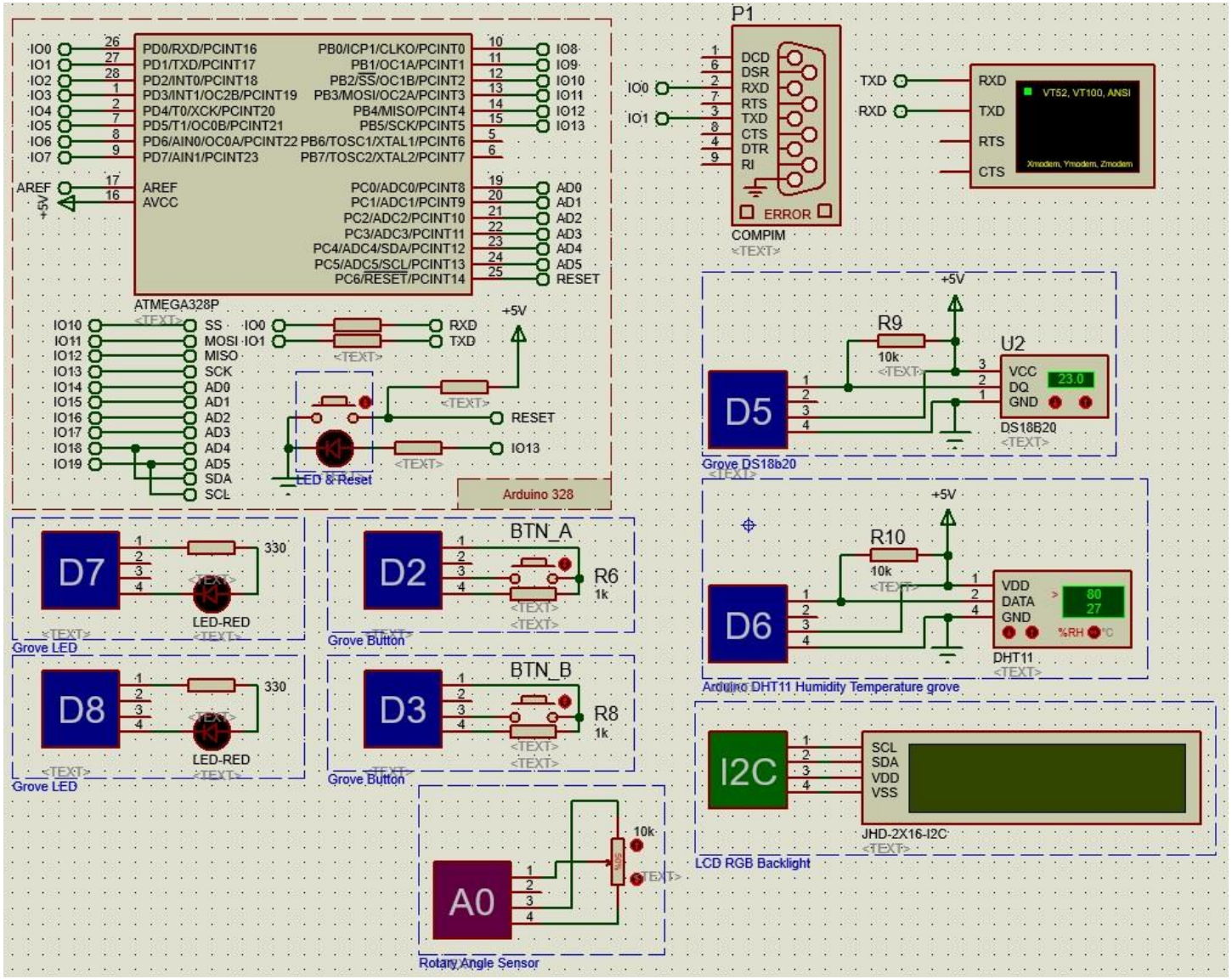
AVIGNON UNIVERSITÉ

Matériel & Câblage Arduino

Composant	Broche Arduino exemple	Type d'E/S	Rôle / Fonctionnalité
LED Builtin	D13	Sortie Numérique	Base de temps / Témoin de fonctionnement (Timer2)
LED Tout-ou-Rien	D7	Sortie Numérique	Commande de commutation à distance depuis Node-RED
LED PWM	D6	Sortie PWM (0-255)	Variateur de luminosité contrôlé par un Slider web
Bouton Poussoir	D2	Entrée (Interruption)	Détection d'appui transmise dans la trame série
DS18b20/DHT11 / Potentiomètre	D4/D5/A0	Entrée Capteur	Mesure environnementale et consigne analogique

L2 Informatique
UE S-E06-0207

Matériel & Câblage Arduino (Schéma structurel)



L2 Informatique
UE S-E06-0207

Pourquoi Choisir Node-RED ?

⚠Approche Web Classique

Architecture complexe : Script Python + Tâche Cron + Base de données MySQL + Serveur Apache/PHP.

Multiplicité de langages : Nécessite C++, Python, SQL, PHP, HTML/CSS et JS.

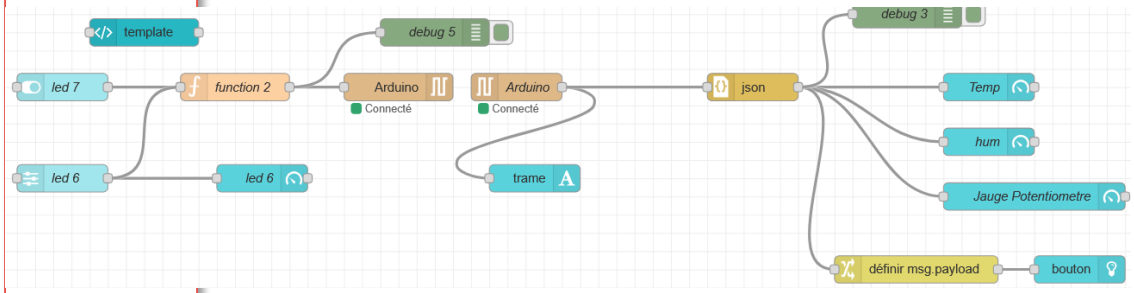
Inconvénient : Temps de développement important pour obtenir un prototype fonctionnel.

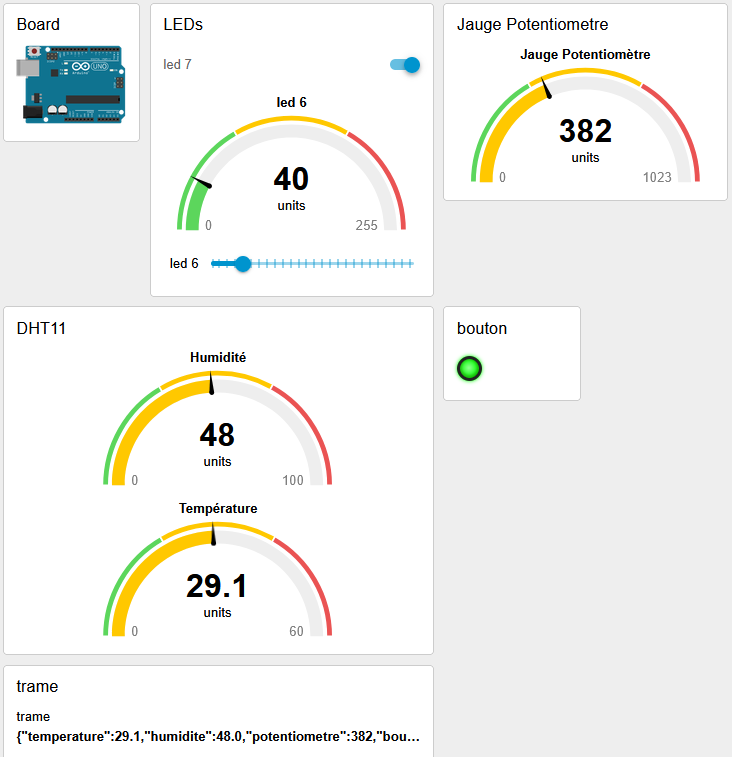
⚡Solution Node-RED

Programmation Visuelle : Connexion de blocs logiciels (« Nodes ») formant un flux (« Flow »).

Dashboard Intégré : Génération automatique d'IHM web sans écriture de code Web/CSS.

Prototypage Rapide : Idéal pour les systèmes embarqués et projets IoT industriels.



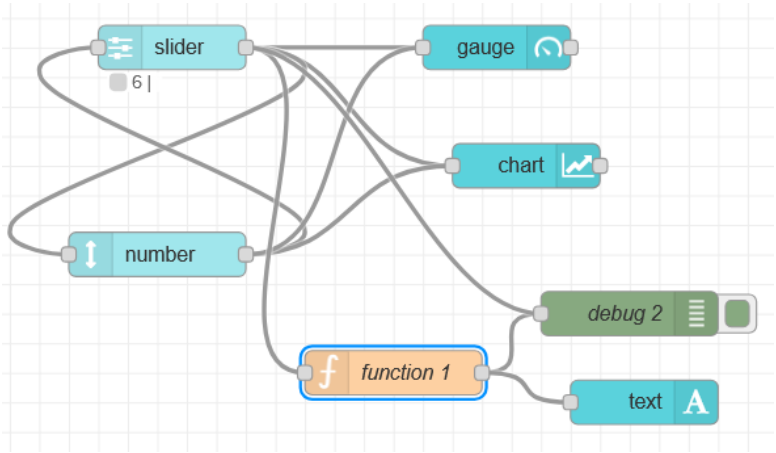


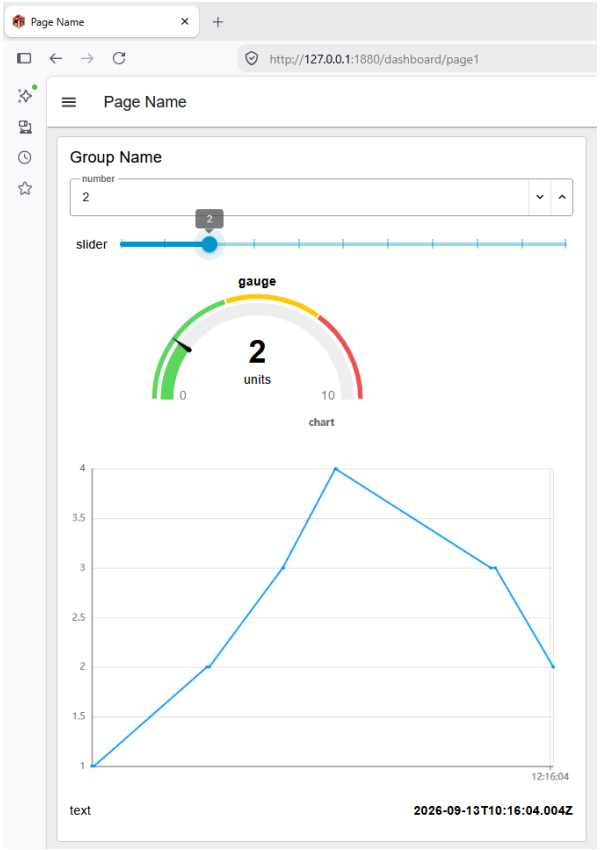
AVIGNON UNIVERSITÉ

Premier Flow & Dashboard Web

Composants du Tableau de Bord

- **Nœud Slider :** Permet d'émettre des valeurs numériques en faisant glisser un curseur.
- **Nœuds Gauge & Chart :** Réceptionnent le payload pour l'afficher sous forme de jauge dynamique ou de graphique temporel.
- **Structure des Onglets :** Organisation IHM en *Tabs* et *Groups* dans l'éditeur.
- **Accès IHM Utilisateur :** Consultation du rendu sur http://localhost:1880/dashboard.





Le Nœud Function & Payload

Structure de Message

Dans Node-RED, les données transitent sous la forme d'un objet JavaScript nommé msg.

La donnée utile véhiculée se trouve dans l'attribut principal : msg.payload.

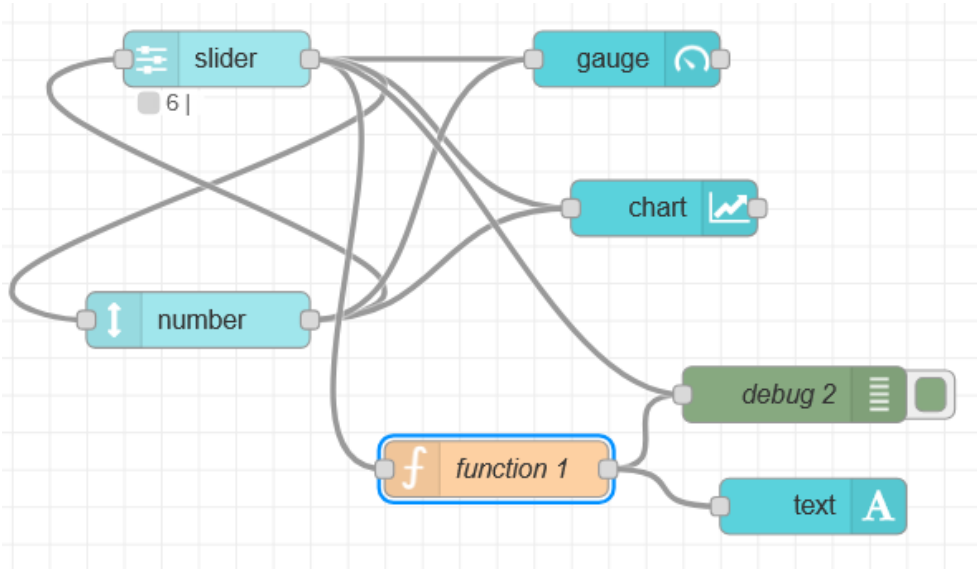
Des propriétés additionnelles peuvent être injectées : msg.topic, msg.timestamp.

Traitements dans Function

Le nœud **Function** permet d'écrire des scripts JS sur mesure pour transformer le message.

```
// Exemple de formatage
var val = msg.payload;
msg.payload = {
   valeur: val,
   status: (val > 50) ? "HIGH" : "LOW"
};
return msg;
```

Les fonctions sont écrites en JAVASCRIPT



Les éléments Node-RED

Élément	Rôle	Syntaxe	Exemple
`msg`	Message qui circule entre les nœuds	`msg.propriété`	`msg.payload`
`msg.payload`	Contenu principal du message	`msg.payload`	`msg.payload = 25;`
`msg.topic`	Identifie le sujet du message	`msg.topic`	`msg.topic = "temperature";`
`msg.payload = ...`	Modifie le contenu du message	`msg.payload = valeur;`	`msg.payload = true;`
`return msg`	Envoie le message vers la sortie de la fonction	`return msg;`	`return msg;`
`flow.get()`	Récupère une donnée mémorisée dans le contexte `flow`	`flow.get("nom")`	`flow.get("trame")`
`flow.set()`	Mémorise une donnée dans le contexte `flow`	`flow.set("nom", valeur)`	`flow.set("trame", trame)`

Dans **Node-RED**, `msg` est le message qui circule entre les nœuds.

Élément	Définition	Exemple
`msg.topic`	Indique **le sujet, le nom ou l'identifiant** de la donnée transportée.	`"temperature"`
`msg.payload`	Contient **la donnée ou la valeur transportée** par le message.	`24.6`

```
msg.topic = "temperature";
msg.payload = 24.6;
return msg;
```

Les chaînes de caractères.

Fonction / opérateur	Rôle	Syntaxe	Exemple
`.toString()`	Convertit en chaîne de caractères	`valeur.toString()`	`msg.payload.toString()`
`.trim()`	Supprime les espaces au début et à la fin	`texte.trim()`	`" true ".trim()` → `"true"`
`+`	Assemble des chaînes	`texte1 + texte2`	`"ID : " + id`
`===`	Compare deux valeurs	`A === B`	`msg.payload === "true"`

```
msg.payload = msg.payload.toString().trim();
```

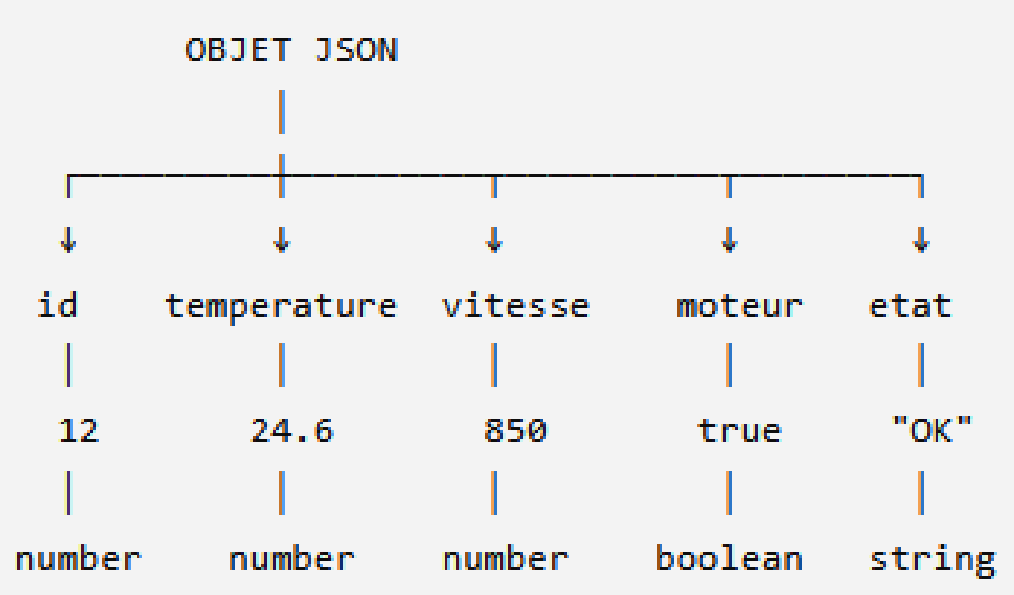
Les conditions.

Élément	Rôle	Syntaxe	Exemple
`if`	Exécute une partie du programme si condition vraie	`if (condition) { }`	`if (temperature > 30)`
`else if`	Teste une autre condition	`else if (condition)`	`else if (x === 0)`
`else`	Cas contraire	`else { }`	`else { ... }`
`===`	Égalité stricte	`A === B`	`x === "true"`
`>`	Supérieur à	`A > B`	`temperature > 30`
`<`	Inférieur à	`A < B`	`vitesse < 500`
`&&`	ET logique	`A && B`	`x > 10 && x < 20`
` ` `		` ` `	OU logique

```
msg.payload = msg.payload.toString().trim();
```

Traitements des données **JSON**

```
{
"id": 12,
"temperature": 24.6,
"vitesse": 850,
"moteur": true,
"etat": "OK«
}
```



Accéder à une donnée

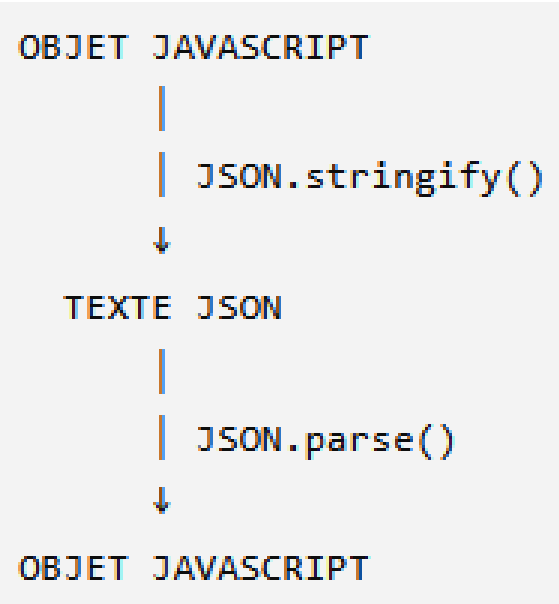
```
doc["id"]          // 12doc
["temperature"] // 24.6
doc["vitesse"]     // 850
doc["moteur"]      // true
doc["etat"]        // "OK"
```

Construire un JSON

```
let trame = {};
trame["temperature"] = 24.6;
trame["vitesse"] = 850;
trame["moteur"] = true;

JSON.stringify(trame);

//transforme l'objet JavaScript en texte JSON.
```



Fonction	Rôle	Syntaxe	Exemple
`JSON.stringify()`	Objet JavaScript → texte JSON	`JSON.stringify(objet)`	`JSON.stringify(trame)`
`JSON.parse()`	Texte JSON → objet JavaScript	`JSON.parse(texte)`	`JSON.parse(message)`

TD 1 — msg.topic et msg.payload

Objectif : comprendre la structure d'un message Node-RED.

```
// On veut envoyer une température
de 24,6 °C.
// Le topic indique le nom de la
donnée.
// Le payload contient sa valeur.


msg._____________ = "temperature";
msg._____________ = 24.6;


// Retourner le message vers la
sortie du nœud
return _________;
```

msg.topic → "temperature"
msg.payload → 24.6

TD 1 — msg.topic et msg.payload

Objectif : comprendre la structure d'un message Node-RED.

```
// On veut envoyer une température de 24,6 ° C.
// Le topic indique le nom de la donnée.
// Le payload contient sa valeur.


msg.topic = "temperature";
msg.payload = 24.6;


// Retourner le message vers la sortie du nœud
return msg;
```

msg.topic → "temperature"
msg.payload → 24.6

TD 2 — Convertir une chaîne en booléen

Objectif : comprendre toString(), trim(), if, === et les booléens.

```
// Convertir le contenu de payload en chaîne de caractères
// puis supprimer les espaces au début et à la fin.

msg.payload = msg.payload._____().______();

// Vérifier si la chaîne obtenue est exactement "true"
if (msg.payload _______ "true")
{
    // Transformer le texte "true" en valeur booléenne true
    msg.payload = _______;
}
// Sinon, vérifier si la chaîne obtenue est "false"
else if (msg.payload _______ "false")
{
    // Transformer le texte "false" en valeur booléenne false
    msg.payload = _______;
}

// Renvoyer le message
return _______;
```

```
" true "
   ↓
toString()
   ↓
" true "
   ↓
trim()
   ↓
"true"
   ↓
if
   ↓
true
```

TD 2 — Convertir une chaîne en booléen

Objectif : comprendre toString(), trim(), if, === et les booléens.

```
// Convertir le contenu de payload en chaîne de caractères
// puis supprimer les espaces au début et à la fin.

msg.payload = msg.payload.toString().trim();

// Vérifier si la chaîne obtenue est exactement "true"
if (msg.payload === "true") {
    // Transformer le texte "true" en valeur booléenne true
    msg.payload = true;
}
// Sinon, vérifier si la chaîne obtenue est "false"
else if (msg.payload === "false") {
    // Transformer le texte "false" en valeur booléenne false
    msg.payload = false;
}

// Renvoyer le message
return msg;
```

TD 3 — Construire un objet JSON

Objectif : comprendre une structure clé/valeur.

```
// Création d'un objet vide.
// Les données seront ensuite ajoutées dans cet objet.

let trame = ________;

// Ajouter l'identifiant.
// "id" est la clé et 12 est la valeur.
trame[________] = 12;

// Ajouter la température.
trame[________] = 24.6;

// Ajouter la vitesse.
trame[________] = 850;

// Ajouter l'état du moteur.
// true est une valeur booléenne : ne pas mettre de guillemets.
trame[________] = ________;

// Placer l'objet dans payload.
msg.payload = trame;

// Renvoyer le message.
return ________;
```

JSON

```
{
  "id": 12,
  "temperature": 24.6,
  "vitesse": 850,
  "moteur": true
}
```

TD 3 — Construire un objet JSON

Objectif : comprendre une structure clé/valeur.

```
// Création d'un objet vide.
// Les données seront ensuite ajoutées dans cet objet.

let trame = {};

// Ajouter l'identifiant.
// "id" est la clé et 12 est la valeur.
trame["id"] = 12;

// Ajouter la température.
trame["temperature"] = 24.6;

// Ajouter la vitesse.
trame["vitesse"] = 850;

// Ajouter l'état du moteur.
// true est une valeur booléenne : ne pas mettre de guillemets.
trame["moteur"] = true;

// Placer l'objet dans payload.
msg.payload = trame;


// Renvoyer le message.
return msg;
```

JSON

```
{
  "id": 12,
  "temperature": 24.6,
  "vitesse": 850,
  "moteur": true
}
```

TD 4 — Transformer un objet en texte JSON

Objectif : comprendre JSON.stringify().

```
let trame = {
    id: 12,
    temperature: 24.6,
    vitesse: 850,
    moteur: true
};


// L'objet trame doit être transformé en texte JSON.
// La fonction permettant de transformer un objet
// JavaScript en texte JSON est JSON._________________().


msg.payload = JSON._________________(________);


// Retourner le message
return ________;
```

TD 4 — Transformer un objet en texte JSON

Objectif : comprendre JSON.stringify().

```
let trame = {
    id: 12,
    temperature: 24.6,
    vitesse: 850,
    moteur: true
};

// JavaScript en texte JSON est JSON.stringify().

msg.payload = JSON.stringify(trame);

// Retourner le message
return msg;
```

TD 5 — Ajouter automatiquement une donnée avec topic

```
trame[msg.topic] = msg.payload;

// Créer un objet vide qui contiendra les données.
let trame = ________;

// Utiliser le topic comme nom de la clé.
// Utiliser le payload comme valeur.
trame[________] = ________;

// Afficher l'objet construit dans payload.
msg.payload = ________;

// Retourner le message.
return ________;
```

TD 5 — Ajouter automatiquement une donnée avec topic

```
trame[msg.topic] = msg.payload;

// Créer un objet vide qui contiendra les données.
let trame = {};

// Utiliser le topic comme nom de la clé.
// Utiliser le payload comme valeur.
trame[msg.topic] = msg.payload;

// Afficher l'objet construit dans payload.
msg.payload = trame;

// Retourner le message.
return msg;
```

TD 6 — Construire progressivement une trame JSON

Objectif : comprendre flow.get(), flow.set(), topic, payload et JSON.stringify().

```
// Récupérer la trame précédemment mémorisée dans le contexte fl
// Si aucune trame n'existe encore, utiliser un objet vide {}.
let trame = flow._________("trame") || ________;
// Ajouter ou modifier une donnée.
//
// Le topic devient la clé du JSON.
// Le payload devient la valeur.
//
// Exemple :
// topic   = "temperature"
// payload = 24.6
//
// donne :
// {
//     "temperature": 24.6
// }
trame[_________________] = _________________;
// Mémoriser la trame modifiée dans le contexte flow.
flow._________("trame", _________);

// Transformer l'objet JavaScript en texte JSON
// et placer le résultat dans msg.payload.
msg.payload = JSON._________________(_________);


// Envoyer le message vers la sortie.


return ________;
```

TD 6 — Construire progressivement une trame JSON

Objectif : comprendre flow.get(), flow.set(), topic, payload et JSON.stringify().

```
// Récupérer la trame précédemment mémorisée dans le contexte flow.
// Si aucune trame n'existe encore, utiliser un objet vide {}.
let trame = flow.get("trame") || {};
// Ajouter ou modifier une donnée.
//
// Le topic devient la clé du JSON.
// Le payload devient la valeur.
//
// Exemple :
// topic   = "temperature"
// payload = 24.6
//
// donne :
// {
//     "temperature": 24.6
// }
trame[msg.topic] = msg.payload;

// Mémoriser la trame modifiée dans le contexte flow.

flow.set("trame", trame);

// Transformer l'objet JavaScript en texte JSON
// et placer le résultat dans msg.payload.
msg.payload = JSON.stringify(trame);

// Envoyer le message vers la sortie.

return msg;
```

TD 7 — Décomposer un JSON reçu

```
// Convertir le texte JSON reçu en objet JavaScript.
// La fonction JSON._____ permet de réaliser cette conversion.
let trame = JSON._____(msg.payload);

// Récupérer la valeur associée à la clé "id".
let id = trame[_____];

// Récupérer la température.
let temperature = trame[_____];

// Récupérer la vitesse.
let vitesse = trame[_____];

// Récupérer l'état du moteur.
let moteur = trame[_____];

// Récupérer l'état sous forme de texte.
let etat = trame[_____];
```


```
{
    "id": 12,
    "temperature": 24.6,
    "vitesse": 850,
    "moteur": true,
    "etat": "OK"
}
```

TD 7 — Décomposer un JSON reçu

```
node.warn(id);
node.warn(temperature);
node.warn(vitesse);
node.warn(moteur);
node.warn(etat);

// Retourner le message.

return _________;
```

JSON

```
{
    "id": 12,
    "temperature": 24.6,
    "vitesse": 850,
    "moteur": true,
    "etat": "OK"
}
```

TD 7 — Décomposer un JSON reçu

```
// Convertir le texte JSON reçu en objet JavaScript.
// La fonction JSON.parse permet de réaliser cette conversion.
let trame = JSON.parse(msg.payload);

// Récupérer la valeur associée à la clé "id".
let id = trame["id"];

// Récupérer la température.

let temperature = trame["temperature"];

// Récupérer la vitesse.

let vitesse = trame["vitesse"];

// Récupérer l'état du moteur.

let moteur = trame["moteur"];

// Récupérer l'état sous forme de texte.
let etat = trame["etat"];

// Afficher les informations dans la console Node-RED.
node.warn(id);
node.warn(temperature);
node.warn(vitesse);
node.warn(moteur);
node.warn(etat);

// Retourner le message.

return msg;
```